

Seneca

SEG800

Digital Signal and Image Processing

Image Transforms

Seneca College

Overview

- Intensity Transformations in Spatial Domain
 - Point/ Pixel operations
 - Neighborhood operation
 - Geometric transformations
 - Histogram Processing

Linear Operations

- Consider an operator $\mathcal{H}$:

$$\mathcal{H}[f(x, y)] = g(x, y)$$

- $\mathcal{H}$ is linear if

$$\begin{aligned}\mathcal{H}[af_1(x, y) + bf_2(x, y)] &= a\mathcal{H}[f_1(x, y)] + b\mathcal{H}[f_2(x, y)] \\ &= ag_1(x, y) + bg_2(x, y)\end{aligned}$$

- Arithmetic (element-wise) operations between two images:

$$s(x, y) = f(x, y) + g(x, y)$$

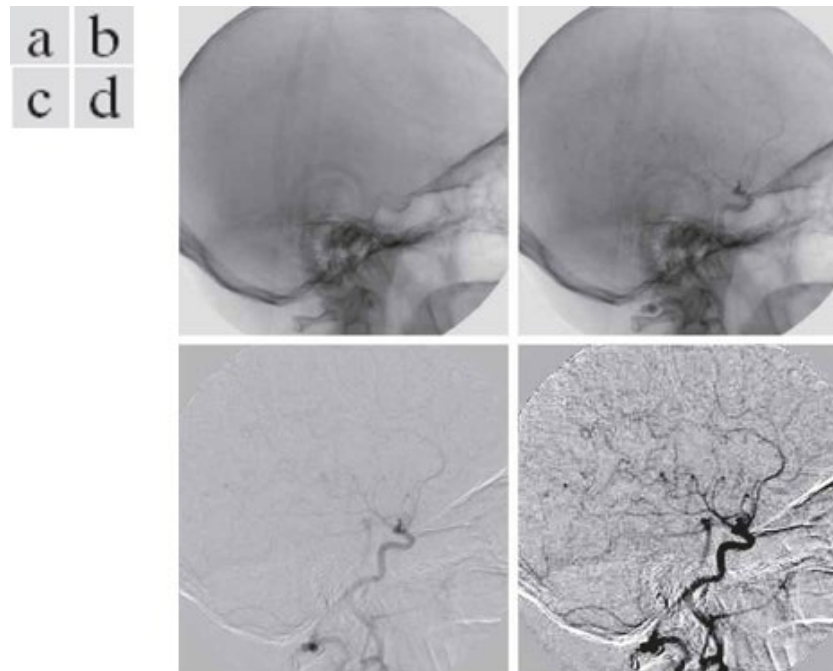
$$d(x, y) = f(x, y) - g(x, y)$$

$$p(x, y) = f(x, y) \times g(x, y)$$

$$v(x, y) = f(x, y) \div g(x, y)$$

Figure 2.32

Digital subtraction angiography. (a) Mask image. (b) A live image. (c) Difference between (a) and (b). (d) Enhanced difference image. (Figures (a) and (b) courtesy of the Image Sciences Institute, University Medical Center, Utrecht, The Netherlands.)



Linear blend (weighted image addition)

- Two input images, *img1* and *img2*
- $dst = \alpha \cdot img1 + \beta \cdot img2 + \gamma$

$\alpha = 1, \beta = 0$



$\alpha = 0.6, \beta = 0.4$



$\alpha = 0.4, \beta = 0.6$



$\alpha = 0, \beta = 1$



Intensity Transformations in Spatial Domain

Spatial Operations

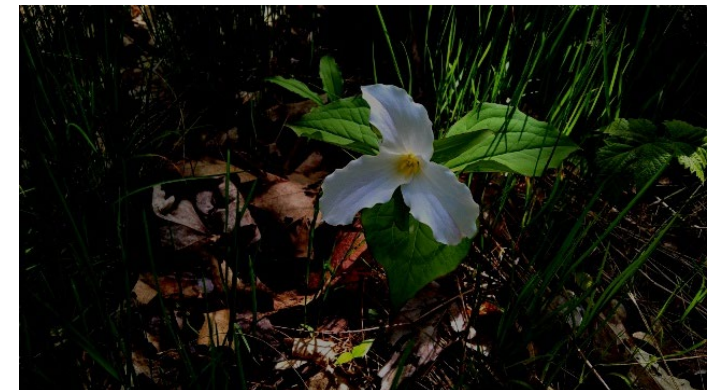
- Single-Pixel (Point operators)
 - The value of each pixel in the output depends only on the value of the same pixel in the input (and possibly some global information or some parameters)
 - Example: brightness adjustment
- Neighborhood Operators:
 - The value of each pixel in the output depends on the value of the pixel and the value of its neighbors in the input
 - Example: Smoothing or blurring
- Geometric spatial transformations
 - Examples: Rotation, translation

Pixel (Point) Transforms

- Assuming one color channel for simplicity
- $I_{\text{out}}(i, j) = f(I_{\text{in}}(i, j))$, for some function f
- Examples:
 - Addition with a constant - Brightness adjustment
$$I_{\text{out}}(i, j) = I_{\text{in}}(i, j) + b$$
 - Multiplication with a constant – Contrast adjustment
$$I_{\text{out}}(i, j) = aI_{\text{in}}(i, j)$$

Addition & Subtraction (scalar)

- Make brighter, e.g., add 50
- Make darker, e.g., subtract 100



Multiply & Divide (scalar)

- Make bright colors brighter- use scale > 1
 - Example: Multiply by 1.4
- Make dark colors darker- use scale < 1
 - Example: Multiple by 0.6



Figure 2.38

(a) An 8-bit image. (b) Intensity transformation function used to obtain the digital equivalent of a “photographic” negative of an 8-bit image. The arrows show transformation of an arbitrary input intensity value z_0 into its corresponding output value s_0 . (c) Negative of (a), obtained using the transformation function in (b).



a b c

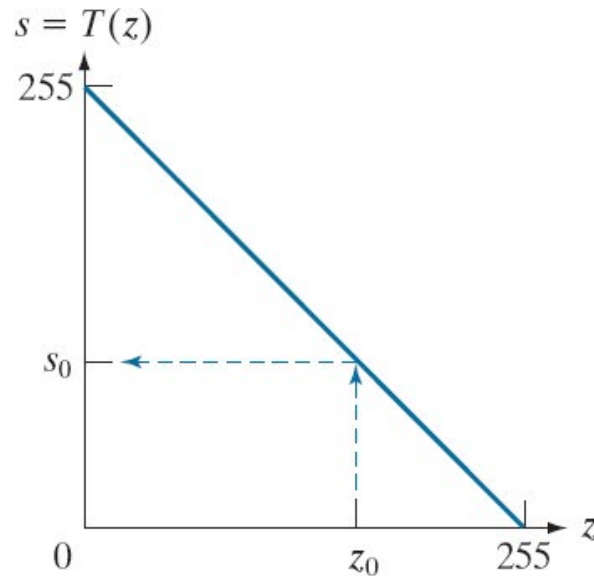


Figure 3.2

Intensity transformation functions. (a) Contrast stretching function. (b) Thresholding function.

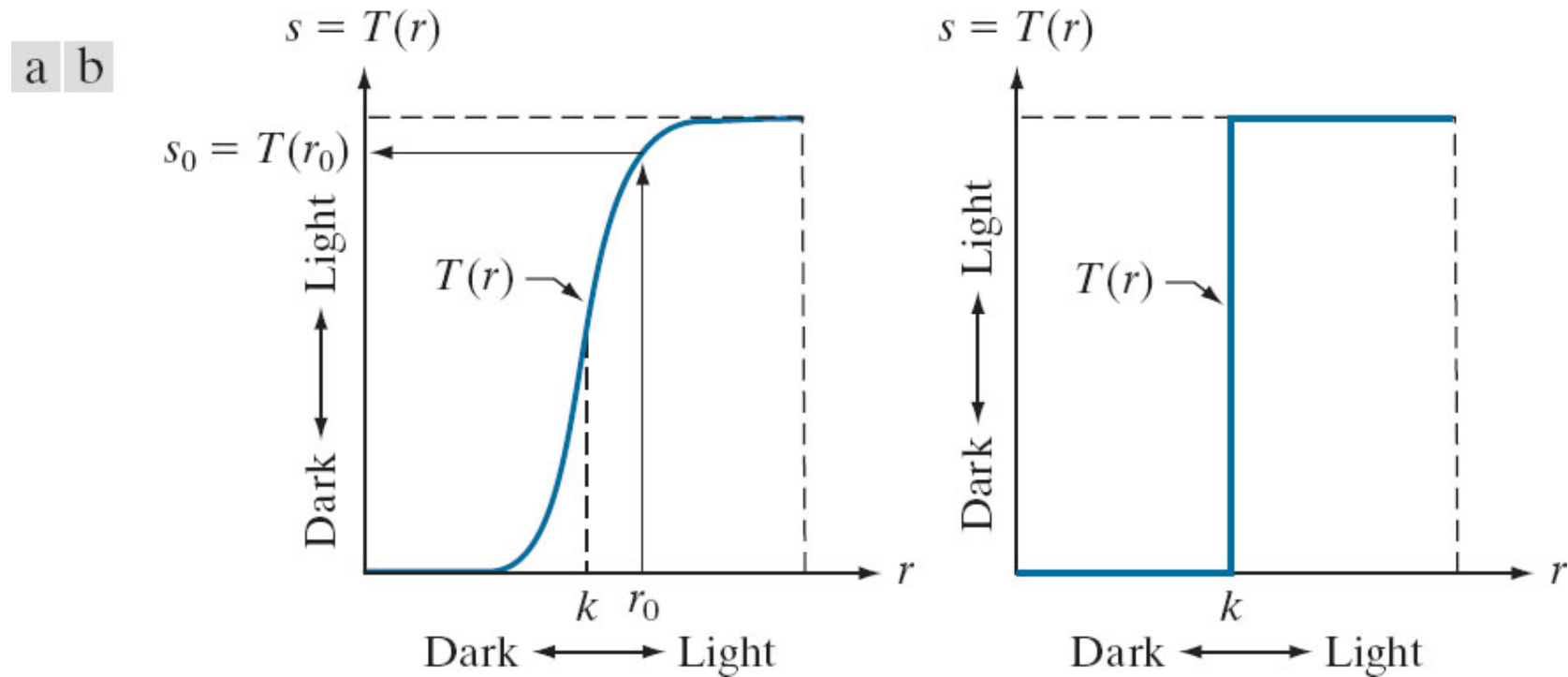


Figure 3.3

Some basic intensity transformation functions. Each curve was scaled **independently** so that all curves would fit in the same graph. Our interest here is on the **shapes** of the curves, not on their relative values.

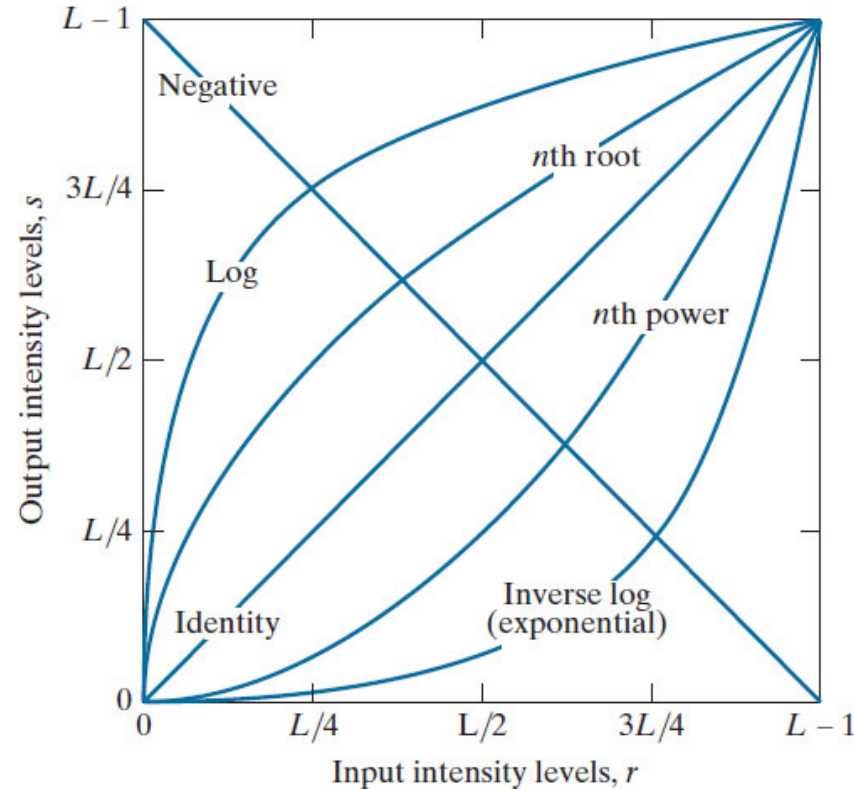


Figure 3.6

Plots of the gamma equation $s = cr^\gamma$ for various values of γ ($c = 1$ in all cases). Each curve was scaled independently so that all curves would fit in the same graph. Our interest here is on the shapes of the curves, not on their relative values.

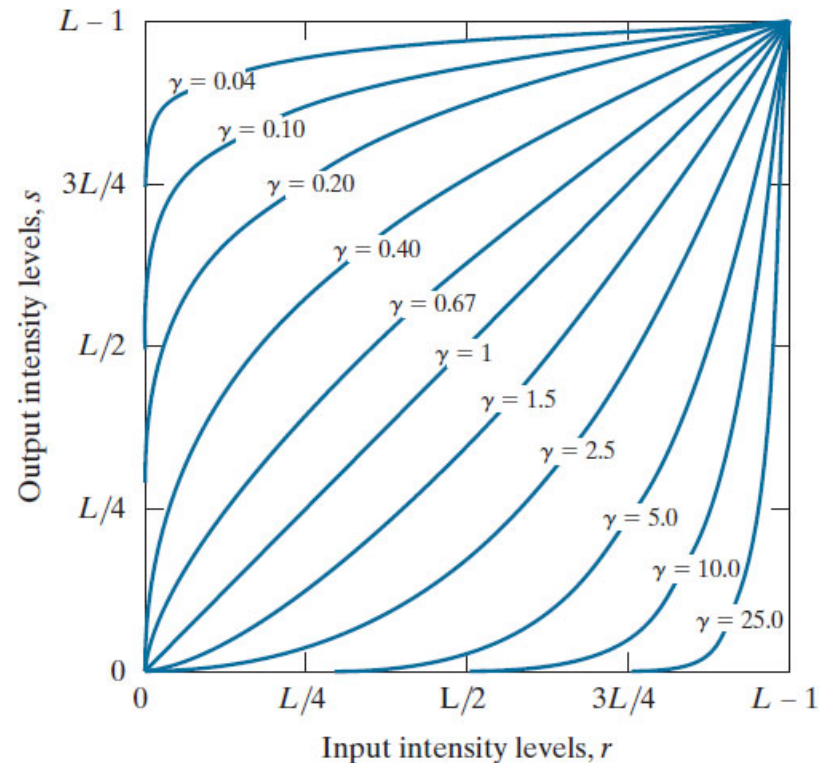
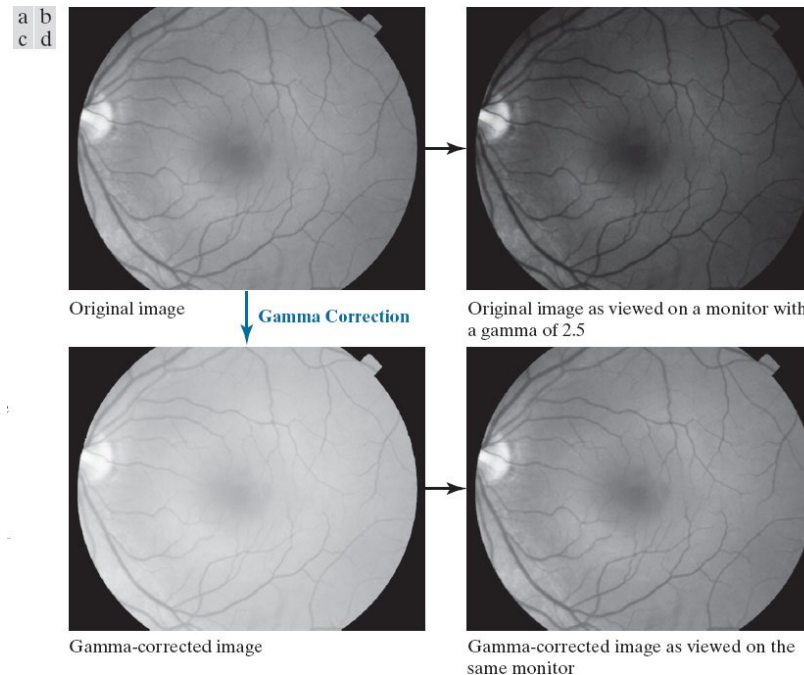


Figure 3.7

(a) Image of a human retina. (b) Image as it appears on a monitor with a gamma setting of 2.5 (note the darkness). (c) Gamma-corrected image. (d) Corrected image, as it appears on the same monitor (compare with the original image). (Image (a) courtesy of the National Eye Institute, NIH)



Neighborhood Operations

- S_{xy} : Neighborhood of pixel at (x,y)
- Size of neighborhood: $m \times n$
- Example: Averaging (or smoothing or local blur)

$$g(x, y) = \frac{1}{mn} \sum_{(r,c) \in S_{xy}} f(r, c)$$

Figure 3.1

A 3×3 neighbourhood about a point (x_0, y_0) in an image. The neighborhood is moved from pixel to pixel in the image to generate an output image. Recall from Chapter 2 that the value of a pixel at location (x_0, y_0) is $f(x_0, y_0)$, the value of the image at that location.

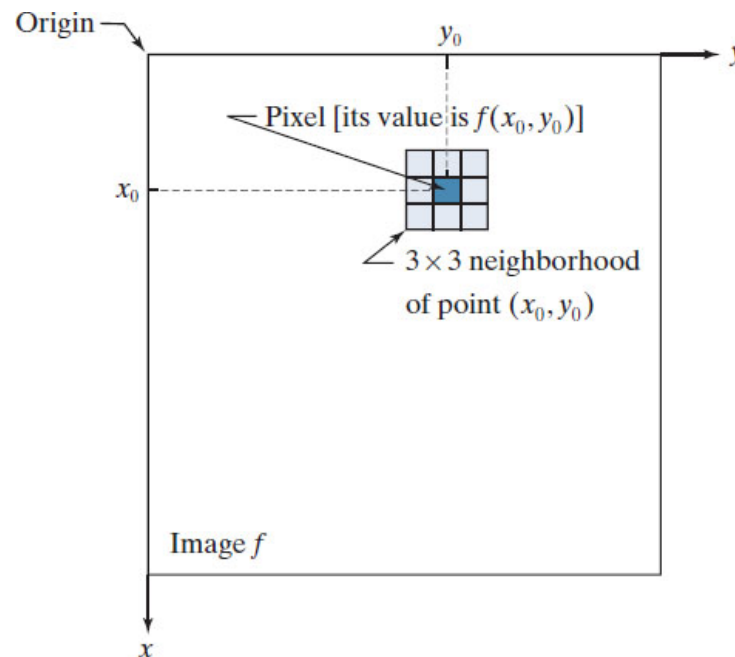
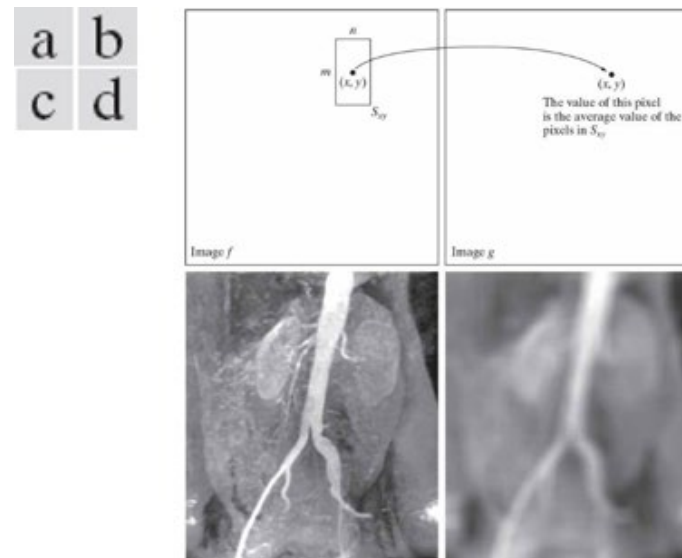


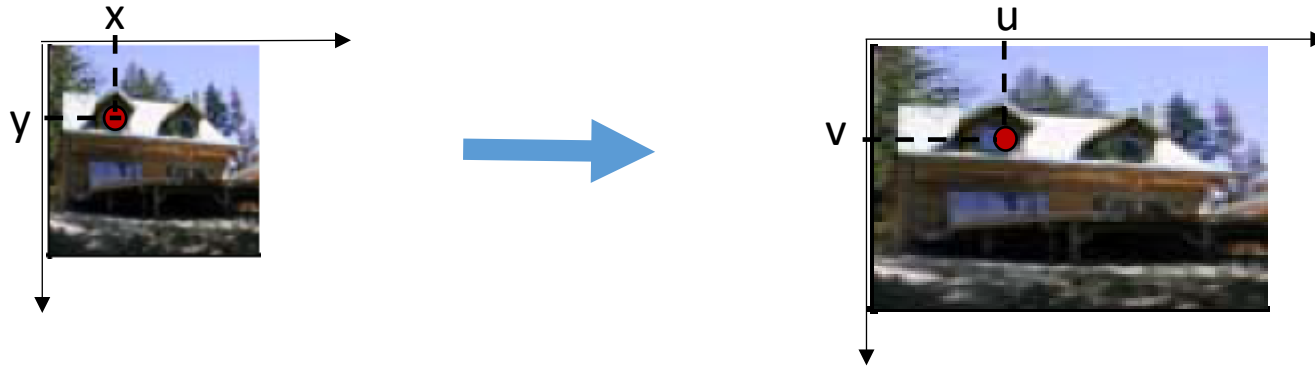
Figure 2.39

Local averaging using neighbourhood processing. The procedure is illustrated in (a) and (b) for a rectangular neighborhood. (c) An aortic angiogram (see Section 1.3). (d) The result of using Eq. (2-43) with $m = n = 41$. The images are of size 790×686 pixels. (Original image courtesy of Dr. Thomas R. Gest, Division of Anatomical Sciences, University of Michigan Medical School.)



Geometric Transformations

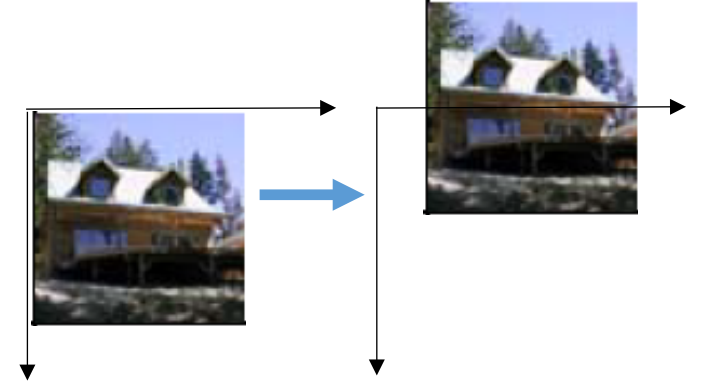
- A pixel in the source image at location (x,y) is mapped to location (u,v) in the destination image



Examples of Geometric Transformation

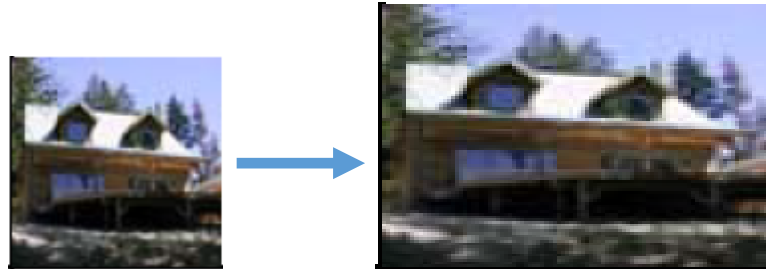
- Translation – pixels move in the same direction

- $u = x + t_x$
 - $v = y + t_y$



- Scale or resize

- $u = x * s_x$
 - $v = y * s_y$



- Rotation

- $u = x * \cos \theta - y * \sin \theta$
 - $v = y * \sin \theta + x * \cos \theta$



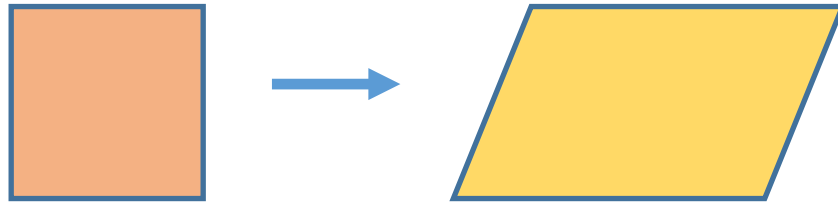
Examples of Geometric Transformation (cont.)

- Shear

- $u = x + y * sh_x$

- $v = y + x * sh_y$

- If $sh_y = 0$



Matrix Notation for Affine Transformations

- **Affine:**

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Translation:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Scale/ Resize:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Rotation:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Shear:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

An equivalent notation

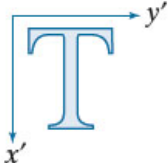
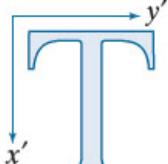
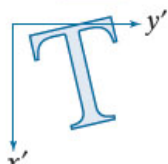
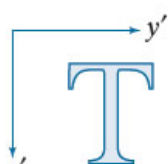
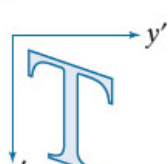
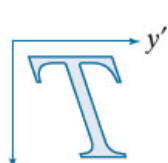
- This is an equivalent matrix notation for affine transforms:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

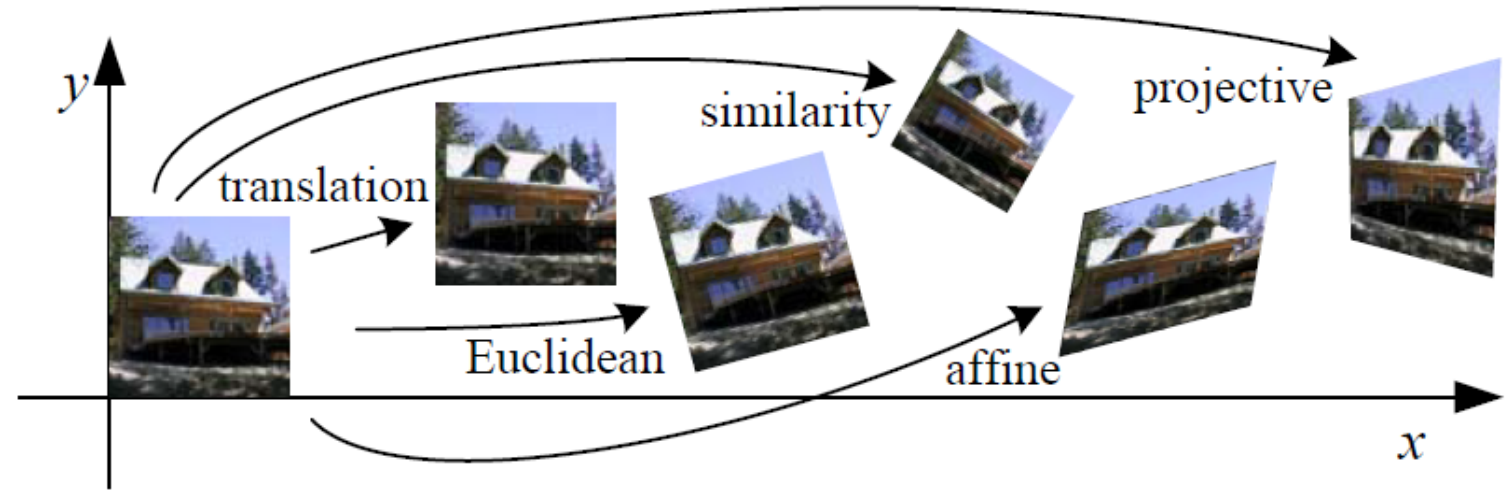
- In general, a transform can be modeled by a 3x3 matrix, called the **homography** matrix.
 - Example: Perspective or projective transforms

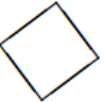
Table 2.3

Affine transformations based on Eq. (2-45).

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \\ y' &= y \end{aligned}$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= c_x x \\ y' &= c_y y \end{aligned}$	
Rotation (about the origin)	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta \end{aligned}$	
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned}$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x + s_v y \\ y' &= y \end{aligned}$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \\ y' &= s_h x + y \end{aligned}$	

2D Geometric Image Transformations



Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} I & & t \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} R & & t \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} sR & & t \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Original



Perspective



Rotation and scale



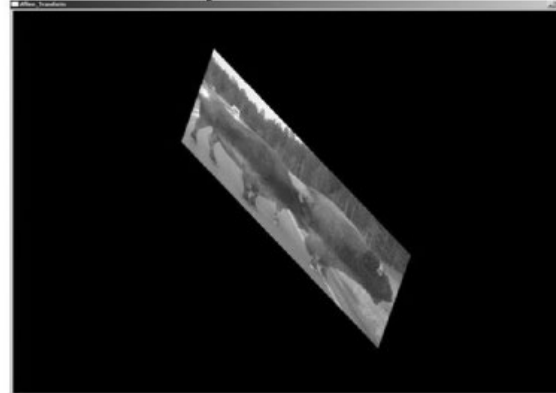
Affine warp



Affine scale



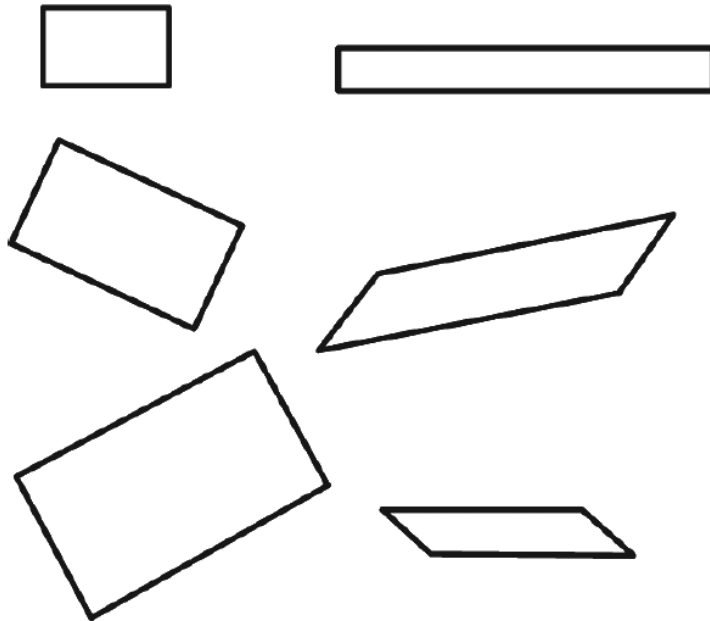
Rotation warp and scale



Affine (2x2)



Parallelograms



Perspective (3x3) (or "Homography")



Trapezoids

(Includes all of Affine)

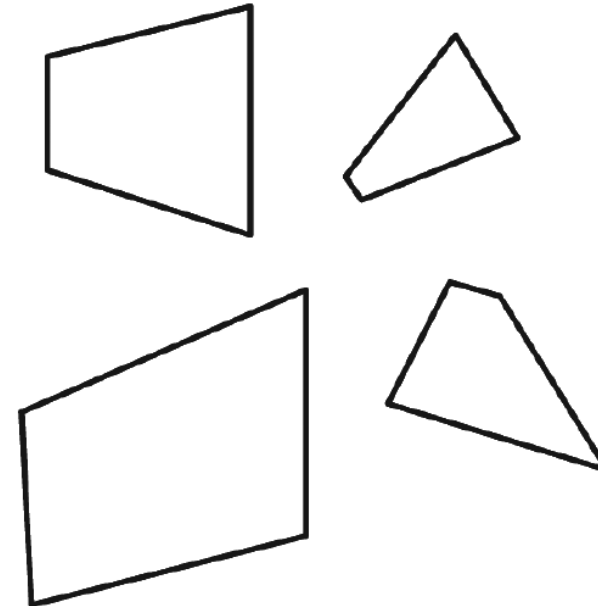


Figure 2.40

(a) A 541×421 image of the letter T. (b) Image rotated -21 degree using nearest-neighbor interpolation for intensity assignments. (c) Image rotated -21 degree using bilinear interpolation. (d) Image rotated -21 degree using bicubic interpolation. (e)-(h) Zoomed sections (each square is one pixel, and the numbers shown are intensity values).

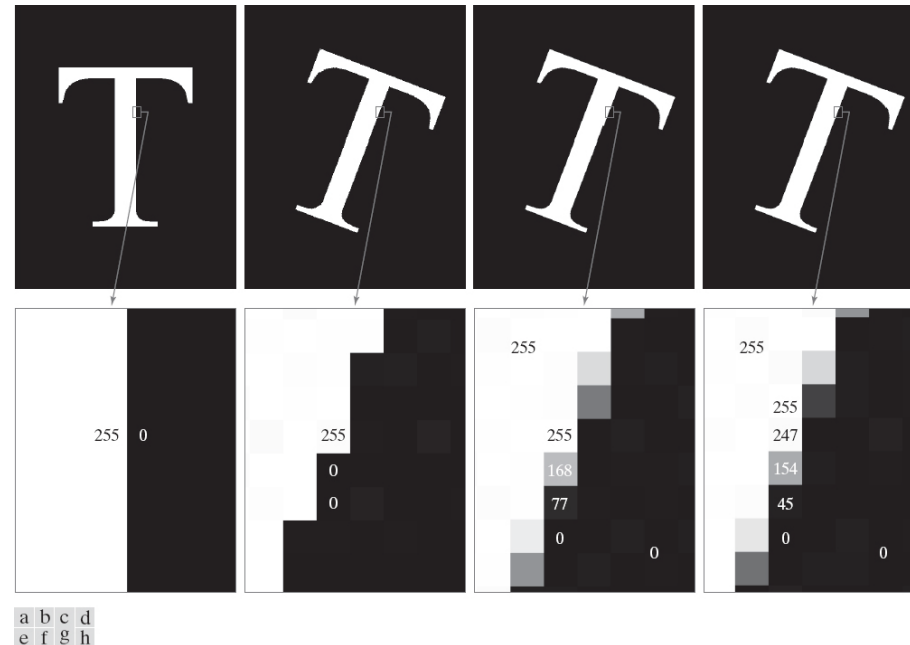
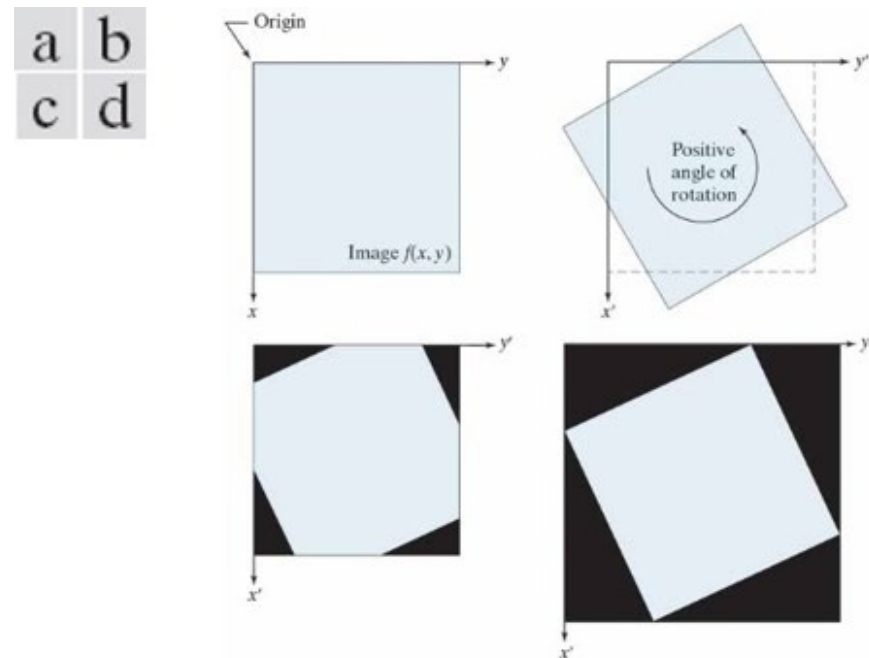


Figure 2.41

(a) A digital image. (b) Rotated image (note the counterclockwise direction for a positive angle of rotation). (c) Rotated image cropped to fit the same area as the original image. (d) Image enlarged to accommodate the entire rotated image.



Domain Transforms

- Some image processing tasks are performed in a transfer domain (e.g., Fourier).
- 2D Linear Transform:

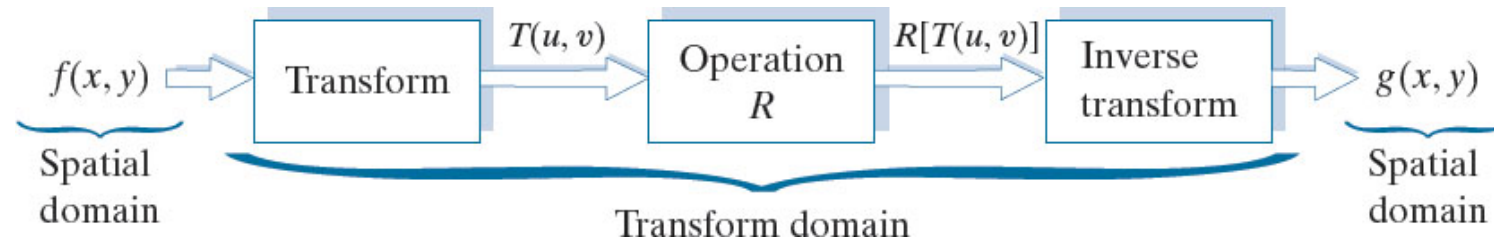
$$T(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) r(x, y, u, v)$$

$$f(x, y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} T(u, v) s(x, y, u, v)$$

- $f(x, y)$: input image
- (x, y) : spatial variables
- (u, v) : transform variables
- $r(x, y, u, v)$: forward transform kernel
- $s(x, y, u, v)$: inverse transform kernel

Figure 2.44

General approach for working in the linear transform domain.



Discrete Fourier Transform

$$r(x, y, u, v) = e^{-j2\pi(ux/M + vy/N)}$$

$$T(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

$$s(x, y, u, v) = \frac{1}{MN} e^{j2\pi(ux/M + vy/N)}$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} T(u, v) e^{j2\pi(ux/M + vy/N)}$$

Separable and Symmetric Kernels

- A kernel is said to be separable if

$$r(x, y, u, v) = r_1(x, u)r_2(y, v)$$

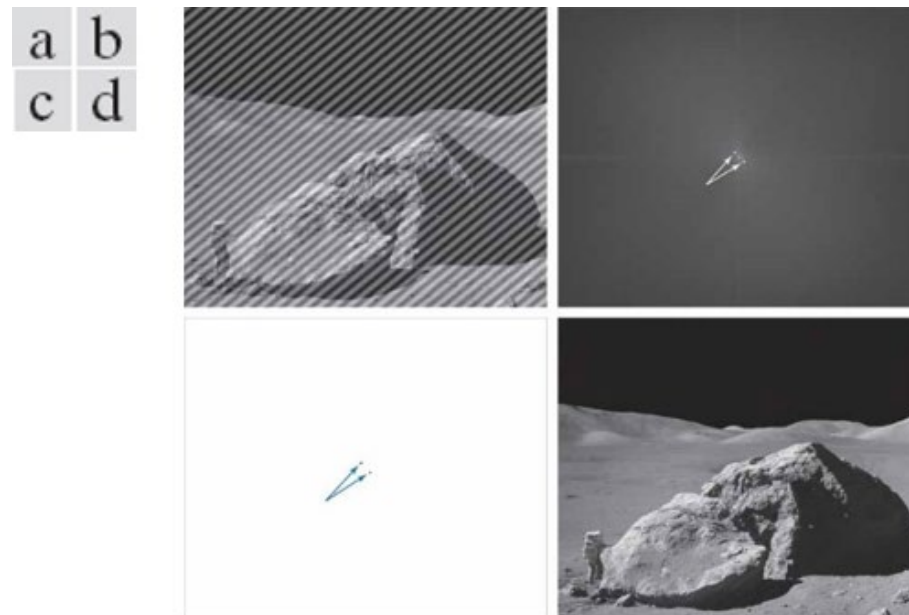
- A separable kernel is said to be symmetric if

$$r(x, y, u, v) = r_1(x, u)r_1(y, v)$$

- Separable and symmetric kernels allow for 2D transforms to be more efficiently computed using 1D transforms.
- Fourier kernels are separable and symmetric.

Figure 2.45

(a) Image corrupted by sinusoidal interference. (b) Magnitude of the Fourier transform showing the bursts of energy caused by the interference (the bursts were enlarged for display purposes). (c) Mask used to eliminate the energy bursts. (d) Result of computing the inverse of the modified Fourier transform. (Original image courtesy of NASA.)



Histogram Processing

Histogram

- The count of the number of pixels at each value

$$h(r_k) = n_k \quad \text{for } k = 0, 1, 2, \dots, L - 1$$

- X axis: intensity values (0 to 255)
- Y axis: number of pixels in the image having that value
- Normalized histogram: Divide by size of image.

$$p(r_k) = \frac{h(r_k)}{MN} = \frac{n_k}{MN} \Rightarrow \sum_{r_k=0}^{L-1} p(r_k) = 1$$



A grey-scale image and the histogram derived from it [3]

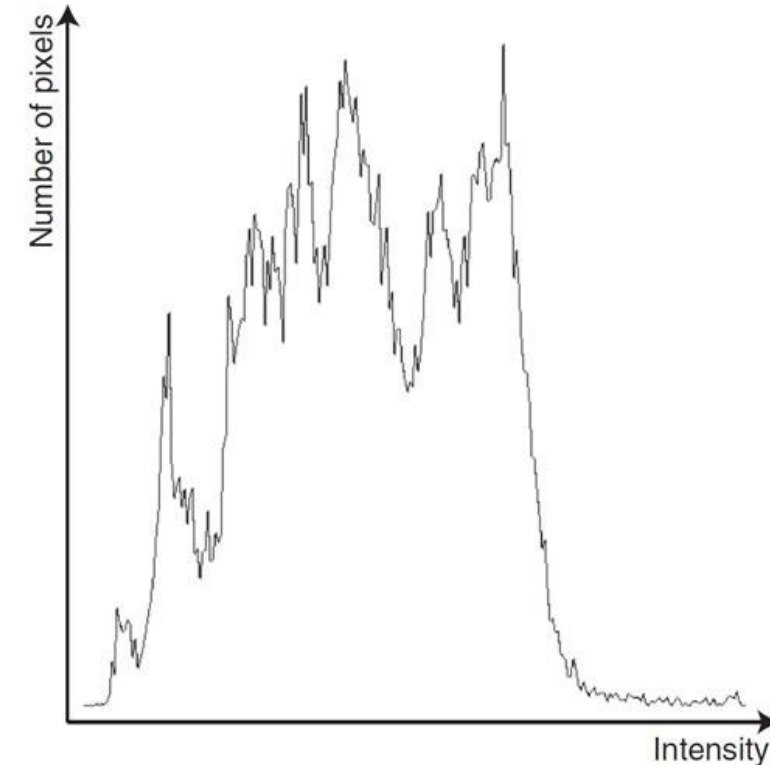
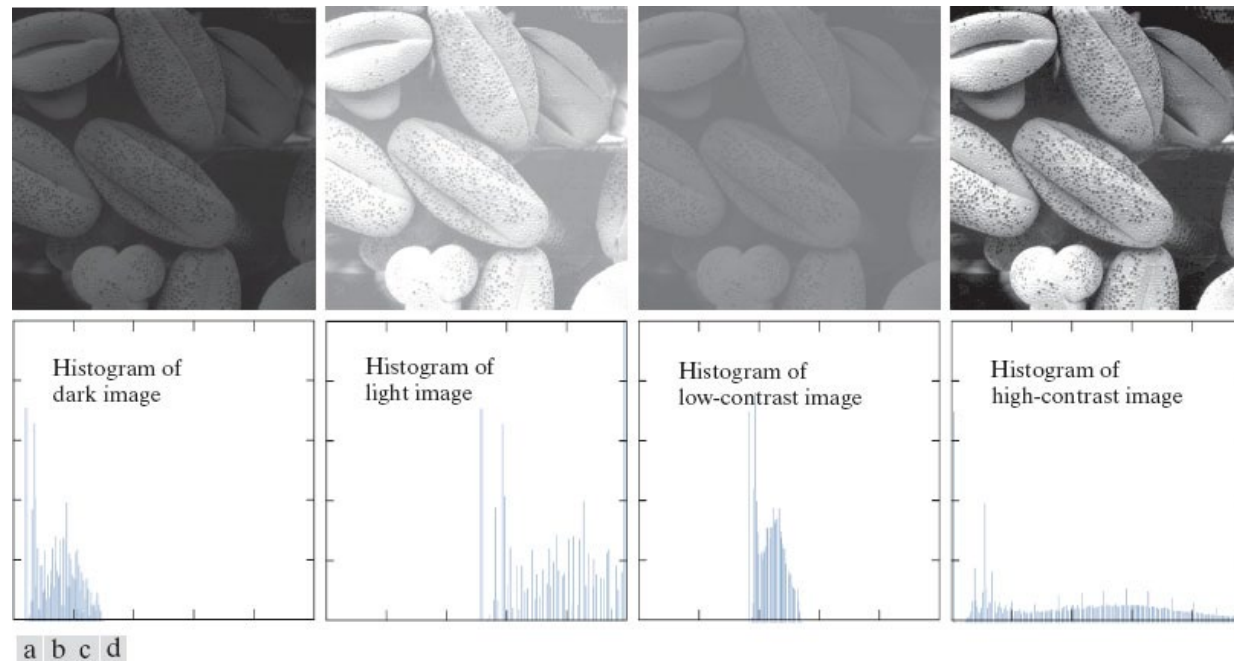


Figure 3.16

Four image types and their corresponding histograms. (a) dark; (b) light; (c) low contrast; (d) high contrast. The horizontal axis of the histograms are values of r_k and the vertical axis are values of $p(r_k)$.

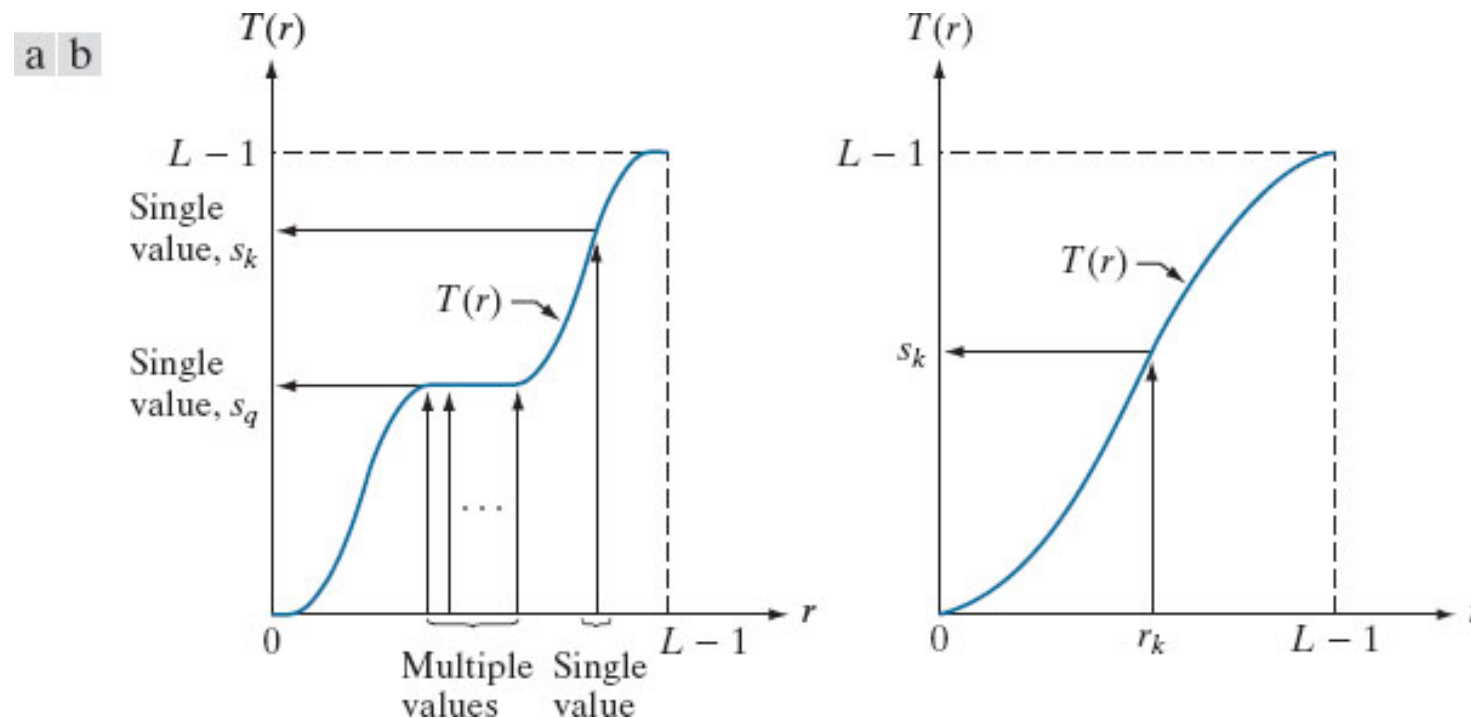


Histogram Processing

- Transforming (mapping) of intensities: $s = T(r) \quad 0 \leq r \leq L - 1$
- Conditions:
 - $T(r)$ is a monotonic increasing function in $0 \leq r \leq L-1$; i.e.,
 $T(r_2) \geq T(r_1)$ for $r_2 > r_1$
 - $0 \leq T(r) \leq L-1$ for $0 \leq r \leq L-1$
- Inverse transformation $r = T^{-1}(s) \quad 0 \leq s \leq L - 1$ exists if and only if
 - $T(r)$ is also **strictly monotonic increasing** in $0 < r < L-1$; i.e.,
 $T(r_2) > T(r_1)$ for $r_2 > r_1$

Figure 3.17

(a) Monotonic increasing function, showing how multiple values can map to a single value. (b) Strictly monotonic increasing function. This is a one-to-one mapping, both ways.



Histogram Equalization

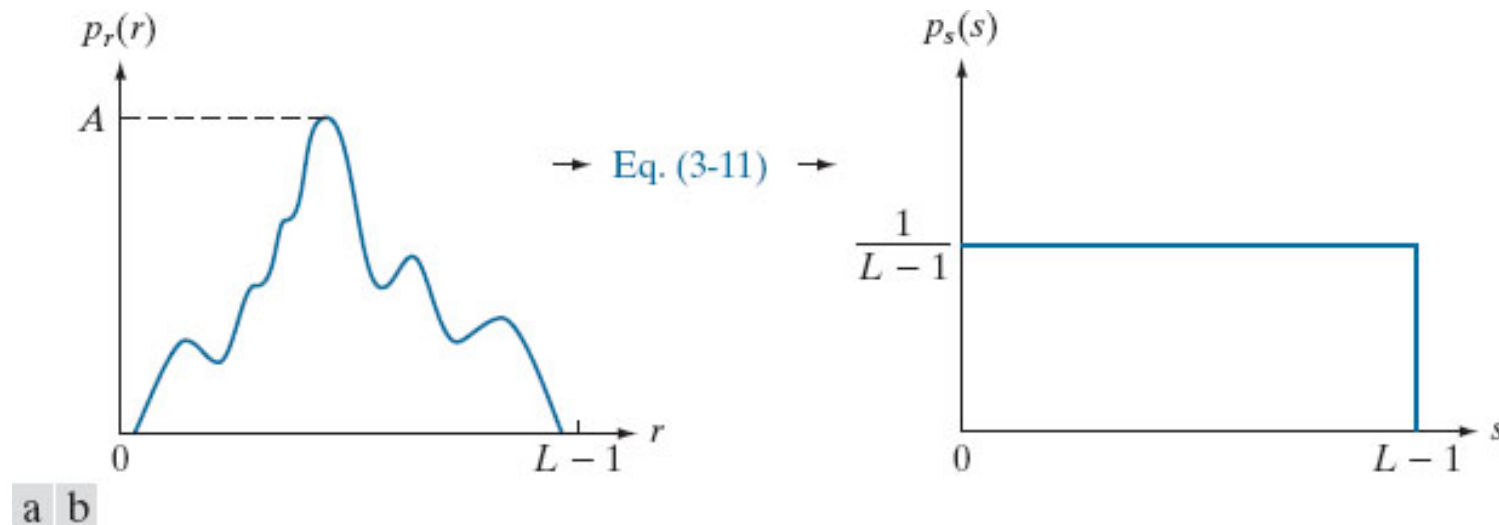
- Transforming the intensities in a way that all intensities have equal probabilities to enhance the image
- Flattening the histogram into a uniform distribution.

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) \quad k = 0, 1, 2, \dots, L - 1$$

- Note that in practice, perfectly flat histograms are rare.

Figure 3.18

(a) An arbitrary PDF. (b) Result of applying Eq. (3-11) to the input PDF. The resulting PDF is always uniform, independently of the shape of the input.



Example: Table 3.1

Intensity distribution and histogram values for a 3-bit, 64×64 digital image.

r_k	n_k	$p_r(r_k) = n_k / MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

Figure 3.19

Histogram equalization. (a) Original histogram. (b) Transformation function. (c) Equalized histogram.

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) \quad k = 0, 1, 2, \dots, L - 1$$

$$\begin{array}{llll} s_0 = 1.33 \rightarrow 1 & s_2 = 4.55 \rightarrow 5 & s_4 = 6.23 \rightarrow 6 & s_6 = 6.86 \rightarrow 7 \\ s_1 = 3.08 \rightarrow 3 & s_3 = 5.67 \rightarrow 6 & s_5 = 6.65 \rightarrow 7 & s_7 = 7.00 \rightarrow 7 \end{array}$$

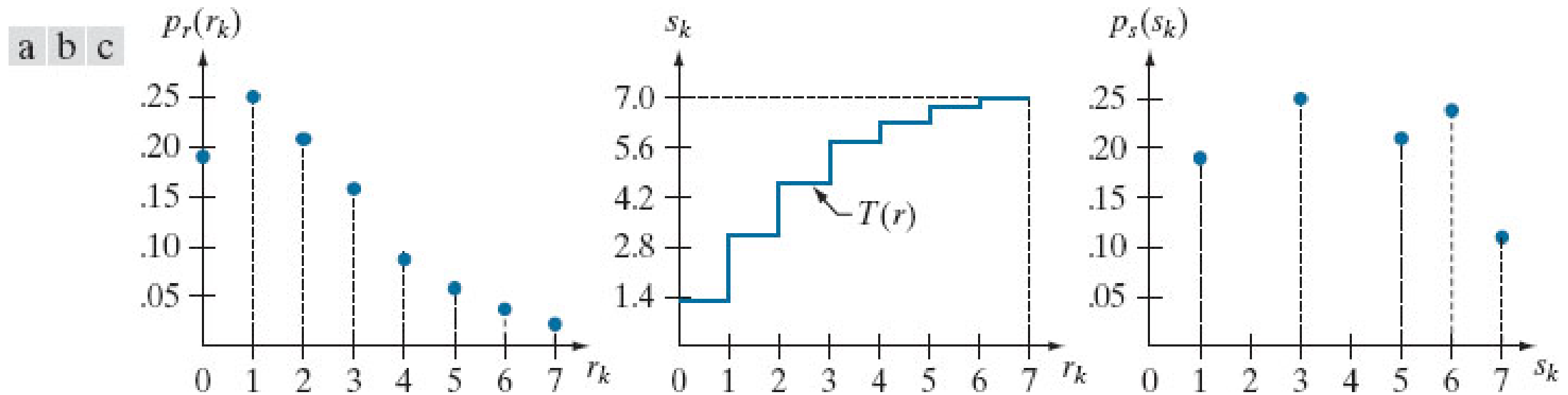


Figure 3.22

(a) Image from Phoenix Lander. (b) Result of histogram equalization. (c) Histogram of image (a). (d) Histogram of image (b). (Original image courtesy of NASA.)

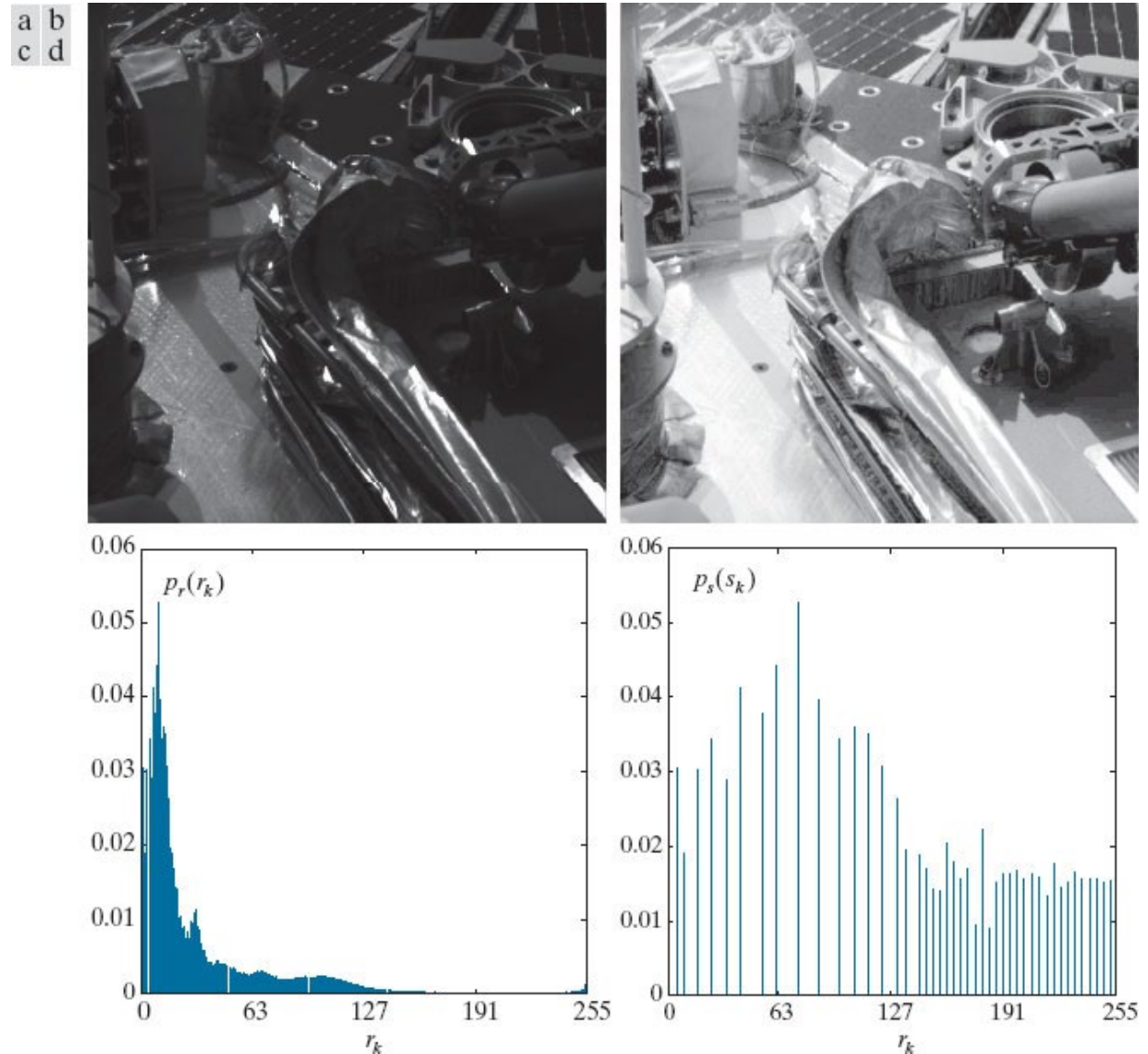


Figure 3.20

Left column: Images from Fig. 3.16. Center column: Corresponding histogram-equalized images. Right column: histograms of the images in the center column (compare with the histograms in Fig. 3.16).

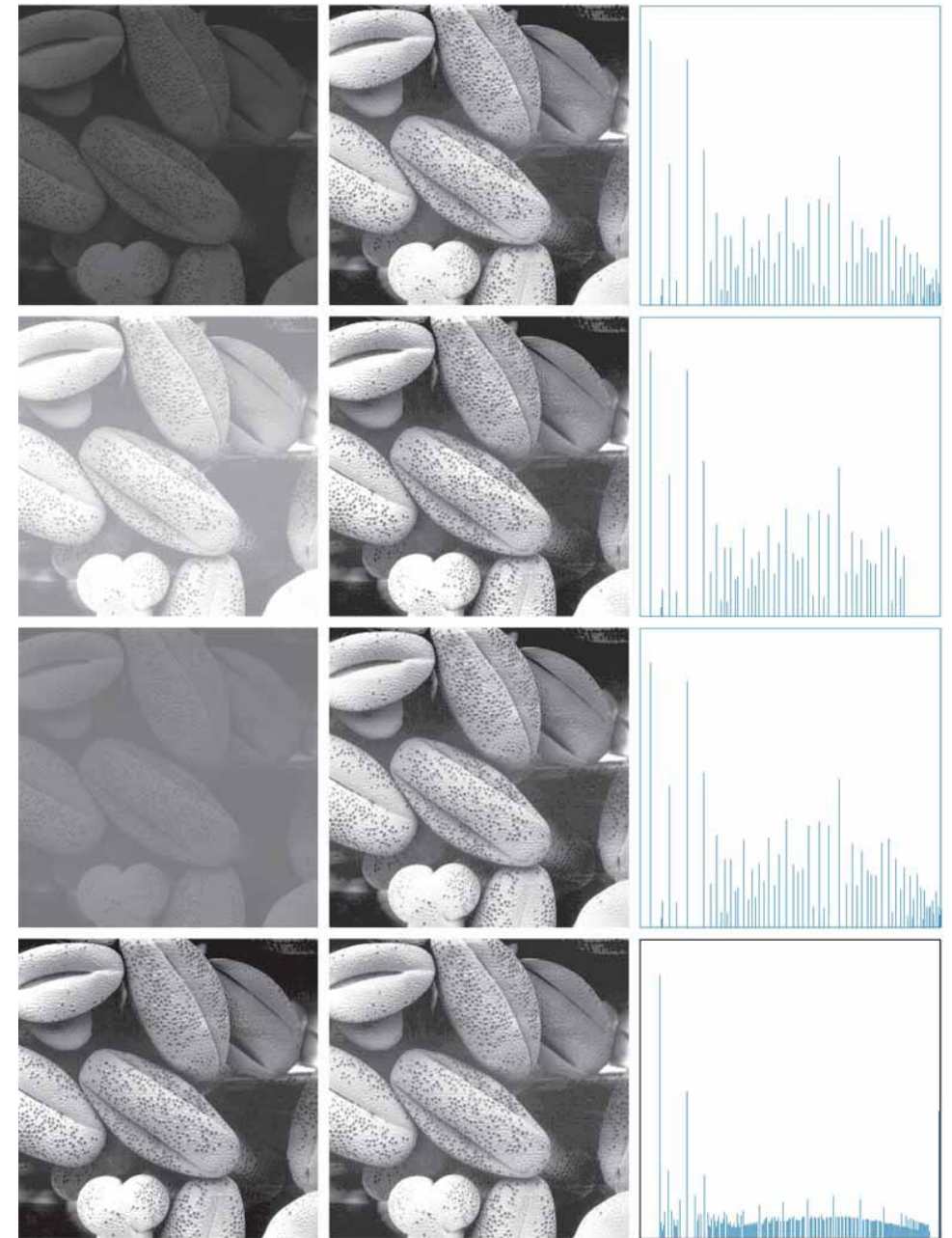
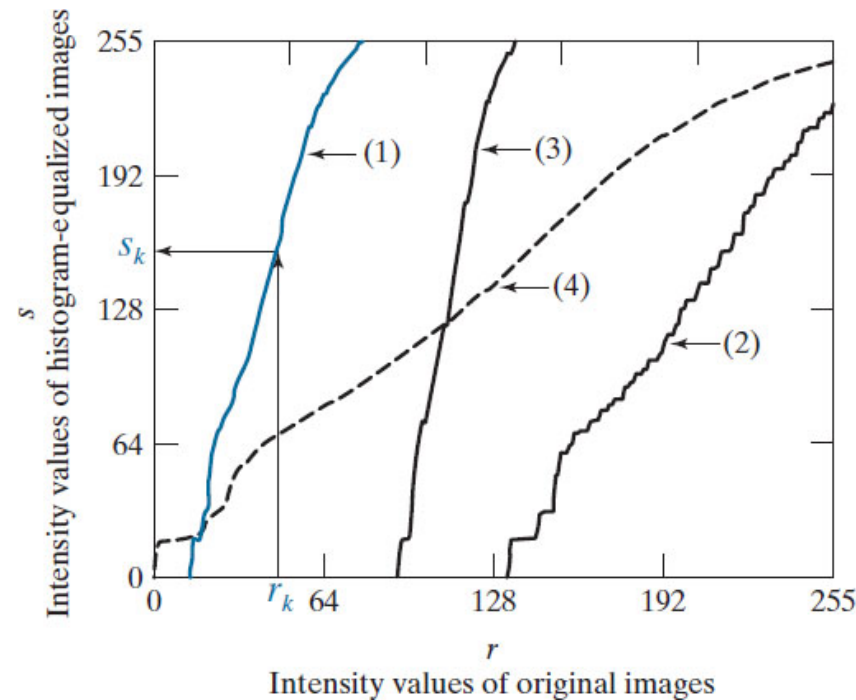


Figure 3.21

Transformation functions for histogram equalization. Transformations (1) through (4) were obtained using Eq. (3-15) and the histograms of the images on the left column of Fig. 3.20. Mapping of one intensity value r_k in image 1 to its corresponding value s_k is shown.



Summary

- **Single-Pixel or Point operator** refers to a transform in which the value of each pixel in the output depends only on the value of the same pixel in the input (and possibly some global information or some parameters).
- **Neighborhood Operator** refers to a transform in which the value of each pixel in the output depends on the value of the pixel and the value of its neighbors in the input
- **Geometric Transformation** transforms the location of pixels (not their intensity/ color values). In affine transformations, parallelism is preserved. Although orientations, lengths, angles and parallelism may all change by projective transformations, straight lines will still be straight lines.

References

- [DIP] Gonzalez, R.C. and Woods, R.E., Digital Image Processing, Pearson, 4th ed., 2018.
- [CV] Computer Vision: Algorithms and Applications, R. Szeliski (<http://szeliski.org/Book>)
- [PICV] Practical introduction to Computer Vision with OpenCV, Kenneth Dawson-Howe, WILEY, 2014. Available through Seneca libraries

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.